COMP 551 Assignment 2: Training and Evaluation Practices

Group 3: Barry Zhang, Maeve Côté, Nick Belev

February 2026

Abstract

We study training and evaluation of logistic regression for spam detection on the UCI Spambase dataset in a small-data regime (5% train, $N = 230$; 95% test), where overfitting is pronounced. We implement mini-batch stochastic gradient descent (SGD) from scratch, monitor cross-entropy loss across epochs, and compare unregularized learning to L2 weight decay (e.g., $\lambda = 10^{-3}$). Using K-fold cross-validation, we tune learning rate, batch size, and epochs, reporting fold means and standard deviations; validation-loss-vs-learning-rate plots show that learning rate and batch size dominate convergence and stability, with larger batches supporting higher learning rates and lower-variance generalization. We then expose the bias–variance trade-off by sweeping λ, obtaining the characteristic U-shaped validation curve and selecting an intermediate optimum ($\approx 10^{-2}$) that minimizes loss and reduces fold-to-fold volatility. Finally, we fit L1-regularized logistic regression in scikit-learn to visualize the regularization path, demonstrating increasing sparsity under stronger penalties and identifying a best-performing mid-range setting ($C \approx 10^2$); the largest-magnitude coefficients highlight intuitive signals such as capital-letter runs for spam and terms like "meeting", "1999", and "George" for non-spam.

1 Introduction

Electronic spam remains a persistent challenge in email communication, motivating the development of reliable machine learning classifiers for automated filtering. In this assignment, we study the UCI Spambase dataset [1], which contains 4,601 email messages represented by 57 continuous features describing word frequencies, character frequencies, and capitalization statistics. The task is binary classification: predicting whether an email is spam (1) or not spam (0).

Logistic regression is a natural baseline model for this problem due to its suitability for binary outcomes and continuous inputs. To make overfitting clearly observable, we intentionally train on only 5% of the data and reserve the remaining 95% as a held-out test set. This small training regime creates a high-variance setting in which regularization and principled model selection become critical.

We implement mini-batch stochastic gradient descent (SGD) with L2 regularization from scratch, use K-fold cross-validation to tune hyperparameters, and sweep the regularization strength to study the bias–variance trade-off. Finally, we analyze L1-regularized logistic regression to visualize the regularization path and sparsity effects.

2 Methods

Preprocessing and Data Split We use the UCI Spambase dataset with 57 continuous features and a binary label. All input features are standardized using training-set statistics (zero mean and unit variance). We initialize a random number generator with seed 2026 to ensure reproducibility.

Following the assignment specification, we split the data once at the beginning into 5% training and 95% test sets using stratified sampling to preserve the spam/non-spam ratio. The test set is held out entirely during model selection and used only for final evaluation.

Logistic Regression with Mini-Batch SGD We implement logistic regression from scratch for binary classification. Given parameters (w, b), predictions are computed as

$$\hat{y} = \sigma(w^\top x + b),$$

where $\sigma(z) = 1/(1 + e^{-z})$ is the sigmoid function.

We minimize the binary cross-entropy loss with L2 regularization:

$$L_{reg}(w, b) = L(w, b) + \lambda \|w\|_2^2.$$

We use the convention $\lambda\|w\|_2^2$, which produces an additional gradient term $2\lambda w$. The bias term b is excluded from regularization.

Mini-batch SGD is used for optimization. At each step, gradients are computed on a batch of size B, and parameters are updated as:

$$w \leftarrow w - \eta \left(\nabla_w L + 2\lambda w\right), \quad b \leftarrow b - \eta \nabla_b L.$$

Training data are shuffled at each epoch.

K-Fold Cross-Validation For hyperparameter tuning (Tasks 2–3), we perform $K = 5$ fold cross-validation on the training split only. For each hyperparameter configuration, we train on $K - 1$ folds and evaluate on the held-out fold, reporting mean cross-entropy and mean accuracy across folds (with standard deviations).

In Task 2, we tune learning rate η, batch size B, and number of epochs. In Task 3, we fix $\eta = 0.1$, $B = 16$, and 200 epochs to ensure strong fitting capacity and sweep λ to study bias–variance behavior.

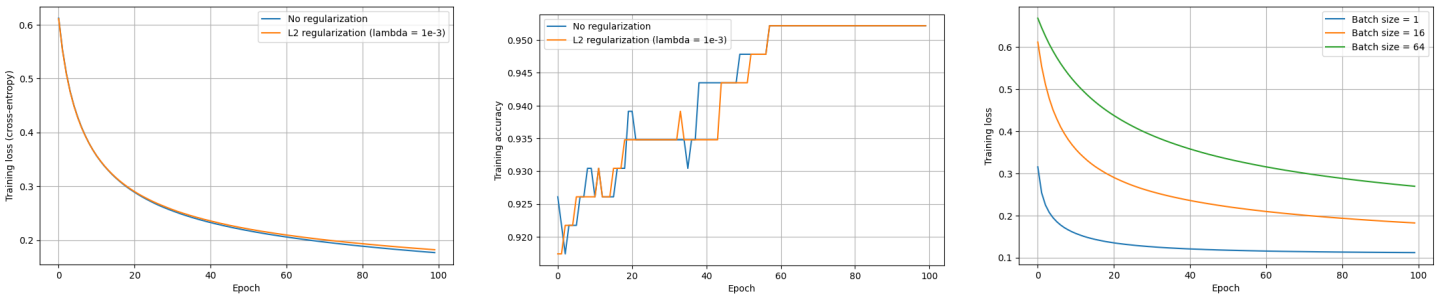
L1-Regularized Logistic Regression For Task 4, we use `sklearn.linear_model.LogisticRegression` with penalty="l1" and solver="saga". scikit-learn parameterizes regularization via $C = 1/\lambda$, where larger C corresponds to weaker regularization. We evaluate a logarithmically spaced grid of C values, record coefficient paths, sparsity levels (number of non-zero weights), and cross-validation accuracy.

3 Results

All models are evaluated using binary cross-entropy (CE) and classification accuracy. Hyperparameters are selected using $K = 5$-fold cross-validation on the (intentionally small) training split, and the final model is retrained on the full training split and evaluated once on the held-out test set.

Task 1: Logistic Regression Trained with SGD We implemented mini-batch SGD for logistic regression from scratch and compared training dynamics with and without L2 regularization. Using a baseline setting of 100 epochs, learning rate 0.01, and batch size 16, the training CE decreases steadily for both models, with a rapid initial drop followed by slower convergence. The non-regularized model achieves a marginally lower training loss after roughly 40 epochs (final gap < 0.01 by epoch 100), which is expected since L2 regularization constrains weight magnitudes and trades slightly higher training loss for improved generalization.

Training accuracy rises quickly for both variants, exceeding 92% within the first few epochs and surpassing 95% by approximately epoch 60. Varying the batch size confirms the expected optimization behavior: smaller batches (e.g., $B = 1$) produce noisier but more frequent updates and achieve the lowest training loss, while larger batches (e.g., $B = 64$) yield smoother updates and slower loss reduction.



(a) Train CE vs. epoch (b) Train acc vs. epoch (c) Batch size effect

Figure 1: Task 1 results for SGD logistic regression, including L2 regularization ($\lambda = 10^{-3}$) and batch size effects.

Task 2: Hyperparameter Tuning with K-Fold Cross-Validation We performed a grid search over 80 hyperparameter combinations: learning rate $\in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}\}$, batch size $\in \{1, 16, 32, 64, 128\}$, and epochs $\in \{50, 100, 150, 200\}$, using $K = 5$-fold cross-validation and fixing the L2 strength to $\lambda = 10^{-3}$ for this task.

When ranked by mean CV loss, the best configuration is learning rate 10^{-3}, batch size 1, and 100 epochs. When ranked by mean CV accuracy, multiple configurations reach a similar peak accuracy (approximately 93%). Among these, learning rate 0.1, batch size 128, and 100 epochs provides the strongest overall trade-off, achieving a low mean CV loss (0.2622) while maintaining high accuracy and comparatively stable fold-to-fold performance.

A sensitivity analysis of validation loss versus learning rate shows that learning rate is one of the most impactful hyperparameters: the mean CV loss can differ by as much as ≈ 0.4 between 10^{-4} and 10^{-1} depending on batch size and epoch setting. Batch size is similarly influential, with large vertical gaps between curves across batch sizes at fixed epochs, indicating substantial changes in generalization performance.

Table 1: Selected cross-validation results for SGD logistic regression (Task 2, $K = 5$, L2 fixed at $\lambda = 10^{-3}$).

Selection Criterion	**LR**	**Batch**	**Epochs**	**Mean CV CE**
Best by mean CV loss	10^{-3}	1	100	*TBD*
Best by mean CV accuracy	10^{-1}	128	100	0.2622

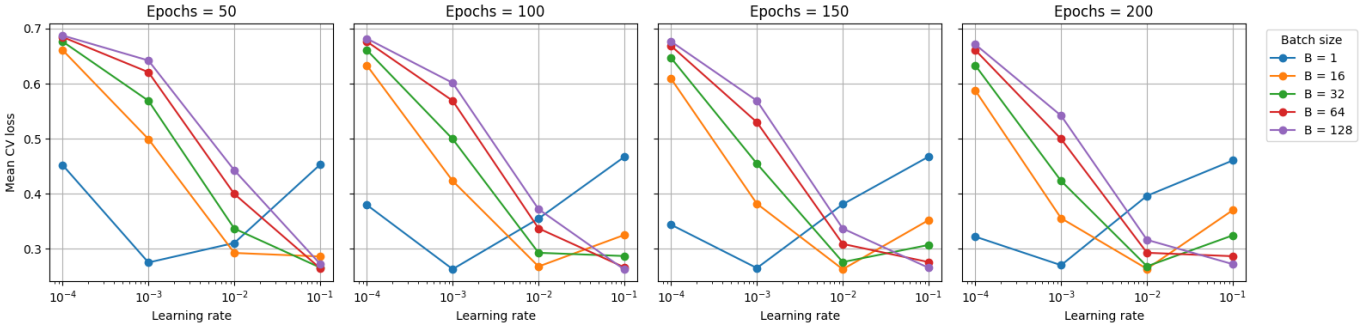


Figure 2: Mean CV CE versus learning rate (log-scale), stratified by epoch count and batch size (Task 2).

Task 3: Bias–Variance Trade-Off via λ Sweep To study the effect of regularization on bias and variance, we swept $\lambda \in \{0, 10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1\}$ using $K = 5$-fold cross-validation with fixed optimization hyperparameters (learning rate 0.1, batch size 16, epochs 200).

For very small λ, the model exhibits a clear train–validation gap in loss, consistent with overfitting (high variance), which is expected given the small training regime (approximately $N = 230$ samples for $d = 57$ features). As λ increases, weight magnitudes shrink and validation performance initially improves. Beyond a certain point, stronger regularization forces the model toward underfitting (high bias), increasing both training and validation loss and

decreasing accuracy.

The validation curve exhibits a U-shaped trend, and the best generalization is achieved near $\lambda = 10^{-2}$, where validation loss is minimized and validation accuracy is maximized before performance degrades in the underfitting regime. Moreover, this also demonstrates that regularization not only prevents overfitting, but also makes the model's performance more stable across different data folds, effectively reducing the variance in the Bias-Variance tradeoff.

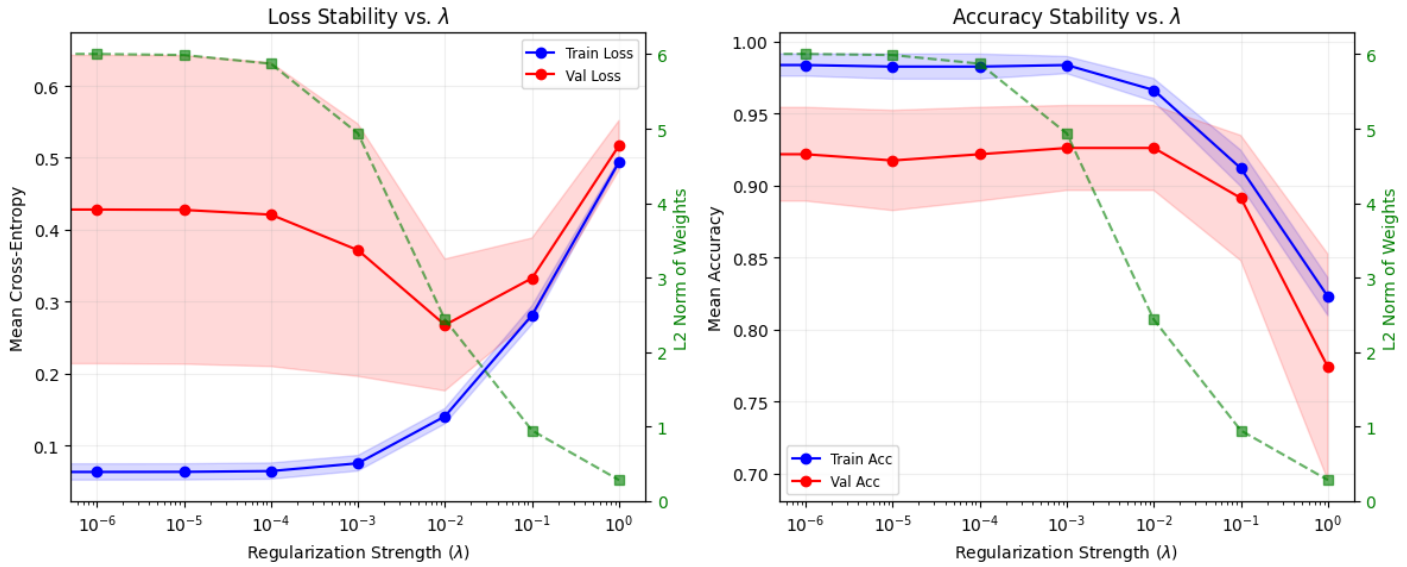


Figure 3: Bias–variance behaviour across L2 strengths: mean train/validation CE and accuracy with ±1 std. bands across folds, with weight norm shown on a secondary axis (Task 3).

After selecting $\lambda^\star = 10^{-2}$, we retrained the model on the full training split and evaluated once on the test set. The final test-set performance is:

$$CE_{test} = 0.4837, \qquad Acc_{test} = 0.8691.$$

Task 4: L1-Regularized Logistic Regression and the Regularization Path Finally, we used scikit-learn to fit L1-regularized logistic regression across a grid of $C \in [10^{-4}, 10^{4}]$ (where C is the inverse regularization strength). As expected, strong regularization (e.g., $C \approx 10^{-2}$) produces a highly sparse model, in this case shrinking all coefficients to zero, while weak regularization (e.g., $C \approx 10^{2}$) retains essentially all features. Cross-validated accuracy peaks at an intermediate value (approximately $C \approx 10^{0}$), indicating a "sweet spot" where L1 regularization performs useful feature selection without overly restricting model capacity.

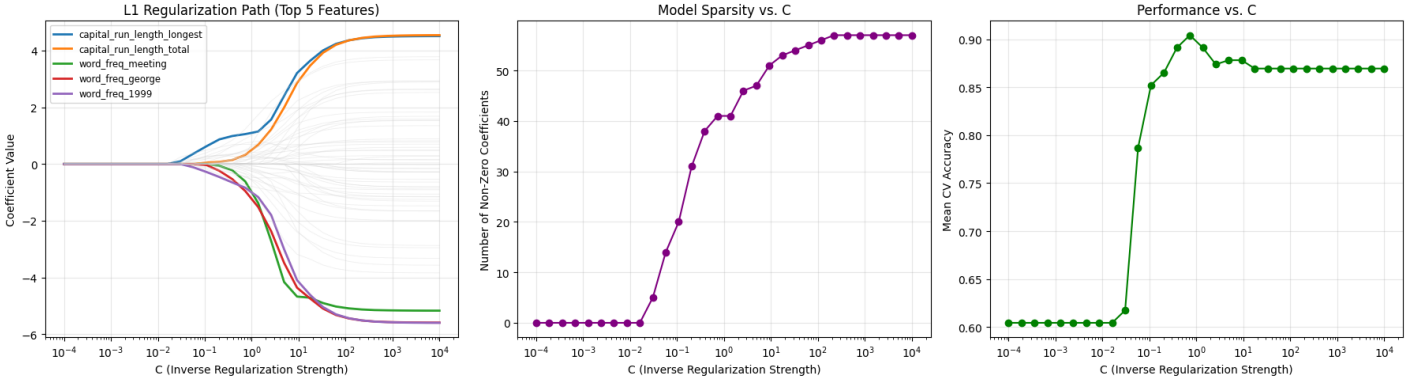


Figure 4: L1 regularization path (top features highlighted), sparsity (non-zero coefficients) vs. C, and mean CV accuracy vs. C (Task 4).

Inspecting the largest-magnitude coefficients at weak regularization highlights interpretable patterns: features related to long runs and high frequencies of capital letters are strong indicators of spam, while terms such as `meeting`, `1999`, and `George` act as indicators of legitimate messages in this dataset.

4 Discussion and Conclusion

Overall, our results show that logistic regression performance in this small-data setting ($N = 230$) is dominated by the interaction between optimization hyperparameters and regularization. The Task 3 λ-sweep exhibits the classic bias–variance trade-off: with small λ the model overfits (high variance) as indicated by a large training–validation loss gap, while large λ shrinks weights too aggressively and underfits (high bias) with high loss on both sets, with the best generalization occurring at an intermediate λ near the minimum of the validation curve and with reduced fold-to-fold volatility. In comparing penalties, L2 regularization smoothly shrinks coefficients and reliably improves generalization, whereas L1 regularizationeenforces sparsity and performs feature selection (improving interpretability) but can be more sensitive to correlated features and the choice of C/λ. Our learning-rate and batch-size experiments further emphasize that convergence and validation loss are highly sensitive to these hyperparameters, with larger batches typically enabling higher learning rates and yielding more stable validation performance. Key limitations include the deliberately tiny training split (higher variance in estimates), a restricted hyperparameter grid, and evaluation primarily via accuracy/loss rather than cost-sensitive metrics (e.g., precision/recall, F1, ROC-AUC). Future work could expand the search strategy (randomized search), add early stopping or learning-rate schedules, evaluate additional metrics and calibration, and compare against stronger baselines (e.g., linear SVM or tree ensembles) or richer feature representations.

5 Statement of Contributions

- **Barry:** Set up the initial project environment and codebase structure; contributed to the K-fold cross-validation pipeline and hyperparameter tuning experiments (Task 2); co-wrote and edited the report.
- **Nick:** Implemented the custom mini-batch SGD logistic regression algorithm (Task 1); collaborated on the hyperparameter grid search and model evaluation (Task 2); co-wrote and edited the report.
- **Maeve:** Conducted the L2 regularization sweep to analyze the bias-variance trade-off (Task 3); generated the L1 regularization path and sparsity visualizations using scikit-learn (Task 4); co-wrote and edited the report.

6 Statement on the Use of LLMs and Other Resources

We used large language model (LLM) tools as writing and development assistants in a limited manner. Specifically, we used ChatGPT and Gemini to improve grammar, clarity, and academic tone, and to suggest LaTeX layout and formatting adjustments. In all cases, we first drafted the report ourselves and then used these tools to refine wording and presentation; the technical content, experimental design, results, and interpretations remain our own. We reviewed and edited all AI-suggested changes to ensure correctness and consistency with our work.

For programming support, we used LLMs for debugging assistance and for generating plotting and visualization code where helpful. Our logistic regression implementation (including mini-batch SGD, the L2 regularization convention and gradient updates, and the cross-validation pipeline) was implemented by hand. The L1-regularized experiments were conducted using scikit-learn as permitted by the assignment instructions.

References

[1] M. Lichman, "Spambase Dataset," UCI Machine Learning Repository, 1999. [Online]. Available: https://archive.ics.uci.edu/dataset/94/spambase